Home IoT 와이어프레임

사용자 화면 · 기기 상세 화면 · 가전 기기 시뮬레이터

공통 개발 구성

프런트엔드 · Next.js

- 화면 경로 (제안): /login, /signup, /devices, /devices/[id], /devices/new
- JWT 를 요청 헤더에 담아 API 호출
- 401 응답이면 로그인으로 이동

백엔드 · Node.js / Express

- REST API: 인증 (JWT), 기기 등록 · 조회 , 기기 명령
- 입력 검증과 오류 응답 (400, 401, 409)
- 명령을 MQTT 로 발행하고 , 기기 상태를 받아 DB 갱신

메시징 · MQTT / 데이터 · PostgreSQL

- 토픽 규칙 (제안): homeiot/{deviceId}/command, homeiot/{deviceId}/state
- users, devices 테이블 (필요하면 기기 이벤트 이력 추가)

제어 흐름

- 사용자 → REST → 서버가 command 발행
- 시뮬레이터 (기기) 가 처리 후 state 발행
- 서버가 DB 갱신 → 화면에 반영

※ 경로 , 토픽 , 테이블 구성은 구현 전 제안 (초안) 이에요

사용자 화면 (Next.js)

로그인



개발 설명

API

- POST /api/auth/login (이메일 , 비밀번호)
- 성공 시 JWT 발급 후 기기 현황으로 이동

동작

- 빈 값 · 이메일 형식 오류는 입력란 아래에 표시
- 실패 메시지는 " 이메일 또는 비밀번호를 확인하세요 " 로 통일
- 토큰이 이미 있으면 기기 현황으로 바로 이동

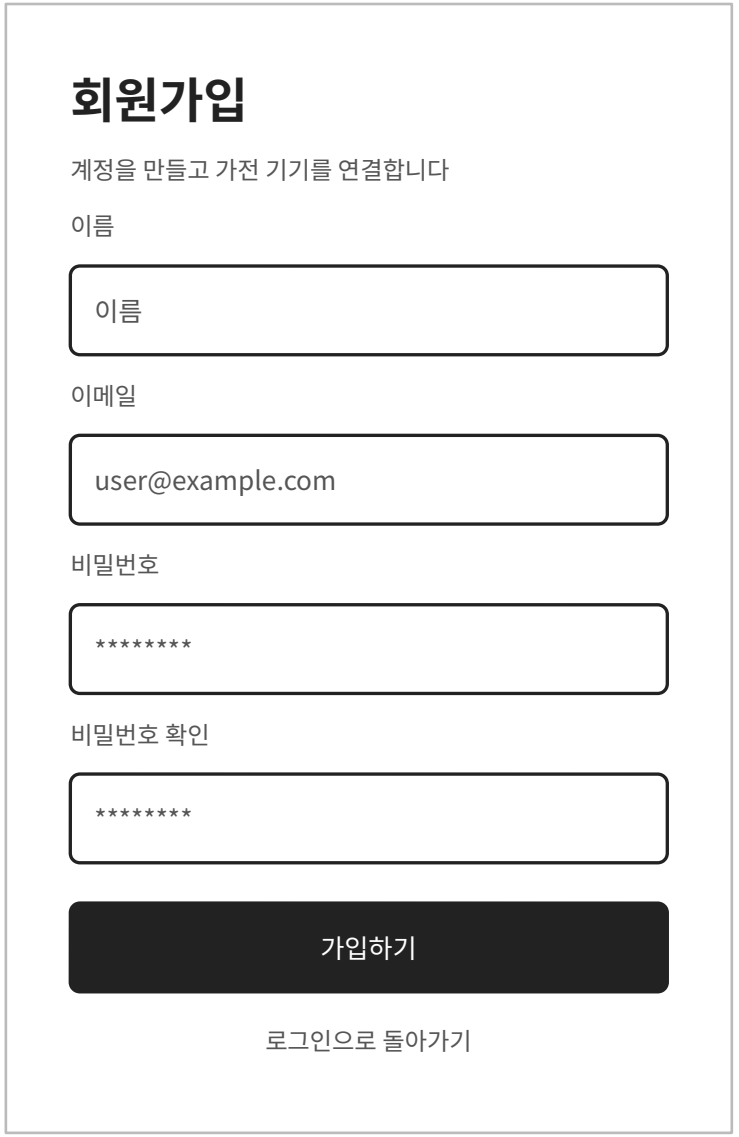
데이터

- users (이메일 , 비밀번호 해시)

※ API 경로와 값의 범위는 구현 전 제안 (초안) 이에요

사용자 화면 (Next.js)

회원가입



개발 설명

API

- POST /api/auth/signup (이름 , 이메일 , 비밀번호)

동작

- 비밀번호 확인 일치 여부는 화면에서 먼저 검사
- 이메일이 이미 있으면 409 → 오류 표시
- 가입 성공 시 로그인 화면으로 이동

데이터

- users 저장 , 비밀번호는 해시로만 저장

※ API 경로와 값의 범위는 구현 전 제안 (초안) 이에요

사용자 화면 (Next.js)

기기 현황 (대시보드)



개발 설명

API

- GET /api/devices (기기 목록과 현재 상태)
- 커튼 : POST /api/devices/{id}/command {action: open | close}
- 엘리베이터 : 같은 API, {action: call}

동작

- 조명 · 에어컨 · 난방기기 · 공기청정기 카드는 누르면 /devices/{id} 상세로 이동
- 등록된 기기가 없으면 기기 등록으로 안내
- 401 이면 로그인으로 , 로그아웃은 토큰 삭제 후 로그인으로

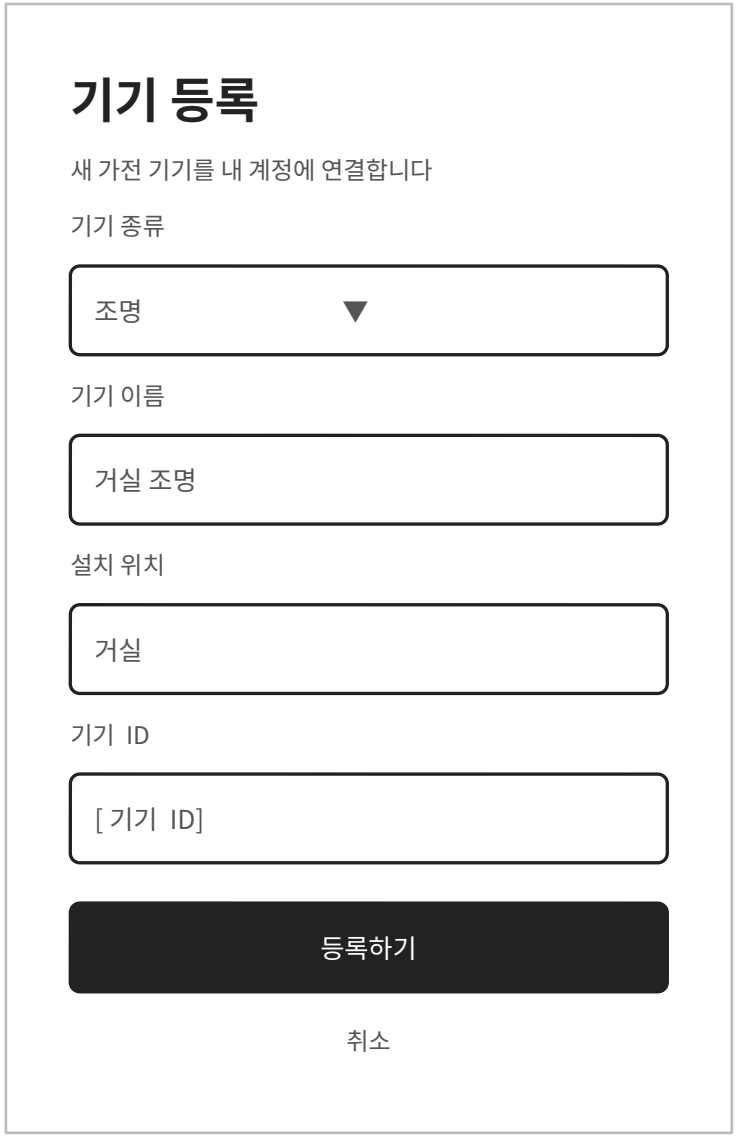
결정 필요

- 상태 갱신 방식 (주기적 조회 / 서버 푸시)

※ API 경로와 값의 범위는 구현 전 제안 (초안) 이에요

사용자 화면 (Next.js)

기기 등록



개발 설명

API

- POST /api/devices (기기 종류 , 이름 , 설치 위치 , 기기 ID)

동작

- 기기 종류는 6 종 중 선택
- 기기 ID 중복은 409, 형식 오류는 400 → 오류 표시
- 등록 후 기기 현황으로 이동

MQTT

- 등록된 기기의 state 토픽을 서버가 구독 시작

데이터

- devices (소유 사용자 , 종류 , 이름 , 위치 , 기기 ID)

※ API 경로와 값의 범위는 구현 전 제안 (초안) 이에요

기기 상세 · 조명



개발 설명

API

- GET /api/devices/{id} (현재 상태 조회)
- POST /api/devices/{id}/command {power, brightness}

MQTT 흐름

- 서버가 homeiot/{id}/command 발행
- 기기가 처리 후 homeiot/{id}/state 발행 → 서버가 DB 갱신
- 화면 상태는 기기가 보고한 값을 기준으로 표시

동작

- 밝기 – /+ 는 10% 씩 (예시), 0~100% 범위
- 요청 중에는 버튼 비활성화 , 실패하면 오류 표시
- ← 버튼은 기기 현황으로 이동

※ API 경로와 값의 범위는 구현 전 제안 (초안) 이에요

기기 상세 · 에어컨



개발 설명

API

- GET /api/devices/{id} (현재 상태 조회)
- POST /api/devices/{id}/command {power, targetTemp}

MQTT 흐름

- 서버가 homeiot/{id}/command 발행
- 기기가 처리 후 homeiot/{id}/state 발행 → 서버가 DB 갱신
- 화면 상태는 기기가 보고한 값을 기준으로 표시

동작

- 희망 온도 – /+ 는 1°C 씩 (예시), 허용 범위 제한 필요
- 요청 중에는 버튼 비활성화 , 실패하면 오류 표시
- ← 버튼은 기기 현황으로 이동

※ API 경로와 값의 범위는 구현 전 제안 (초안) 이에요

기기 상세 · 난방기기



개발 설명

API

- GET /api/devices/{id} (현재 상태 조회)
- POST /api/devices/{id}/command {power, targetTemp}

MQTT 흐름

- 서버가 homeiot/{id}/command 발행
- 기기가 처리 후 homeiot/{id}/state 발행 → 서버가 DB 갱신
- 화면 상태는 기기가 보고한 값을 기준으로 표시

동작

- 희망 온도 – /+ 는 1°C 씩 (예시), 허용 범위 제한 필요
- 요청 중에는 버튼 비활성화 , 실패하면 오류 표시
- ← 버튼은 기기 현황으로 이동

※ API 경로와 값의 범위는 구현 전 제안 (초안) 이에요

기기 상세 · 공기청정기



개발 설명

API

- GET /api/devices/{id} (현재 상태 조회)
- POST /api/devices/{id}/command {power, fanLevel}

MQTT 흐름

- 서버가 homeiot/{id}/command 발행
- 기기가 처리 후 homeiot/{id}/state 발행 → 서버가 DB 갱신
- 화면 상태는 기기가 보고한 값을 기준으로 표시

동작

- 풍량은 약 / 중 / 강 중 하나만 선택 , 선택값 강조
- 요청 중에는 버튼 비활성화 , 실패하면 오류 표시
- ← 버튼은 기기 현황으로 이동

※ API 경로와 값의 범위는 구현 전 제안 (초안) 이에요

시뮬레이터 · 대시보드

가전 기기 시뮬레이터 MQTT 연결됨

조명
현재 상태 : 꺼짐
마지막 명령 : [수신 시각]
상태 변경

에어컨
현재 상태 : 24°C
마지막 명령 : [수신 시각]
상태 변경

난방기기
현재 상태 : 22°C
마지막 명령 : [수신 시각]
상태 변경

공기청정기
현재 상태 : 풍량 중
마지막 명령 : [수신 시각]
상태 변경

커튼
현재 상태 : 열림 40%
마지막 명령 : [수신 시각]
상태 변경

엘리베이터 호출
현재 상태 : 1 층 · 호출 대기
마지막 명령 : [수신 시각]
도착 처리

이벤트 로그
[hh:mm:ss] [토픽] [페이로드]
[hh:mm:ss] [토픽] [페이로드]
[hh:mm:ss] [토픽] [페이로드]

개발 설명

구성
- 별도 Node.js 서비스 , MQTT 클라이언트로 동작
- 기기별로 command 구독 , state 발행
- 서버 DB 에는 직접 접근하지 않음

화면
- MQTT 연결됨 : 브로커 연결 상태
- 상태 변경 : 값을 직접 바꿔 state 발행 (기기를 손으로 조작한 경우 재현)
- 도착 처리 : 호출 수신 후 도착 상태 발행
- 이벤트 로그 : 수신 · 발행 메시지의 시각 , 토픽 , 페이로드

※ API 경로와 값의 범위는 구현 전 제안 (초안) 이에요